Լաբորատոր աշխատանք 5

Կառուցվածքային տիպի զանգվածների օգտագործումը ֆունկցիաներում

Աշխատանքի նպատակ. Ծանոթանալ C++ ծրագրավորման լեզվում կառուցվածքային տիպերի (`struct`) և կառուցվածքների զանգվածների օգտագործմանը: Զարգացնել տվյալների մուտքագրման, ստուգման, որոնման, փոփոխման և հաշվարկների կատարման հմտությունները՝ օգտագործելով առանձին ֆունկցիաներ, պայմաններ և ցիկլեր:

Խնդրի պահանջ. Ստեղծել `Ticket` կառուցվածք՝ `ticketNumber`, `sector`, `row`, `price`, `sold` դաշտերով: Մուտքագրել 15 տոմսի տվյալներ: Ծրագիրը պետք է հաշվի վաճառված տոմսերից ստացված ընդհանուր գումարը, արտածի տրված սեկտորի ազատ տոմսերը, գտնի ամենաթանկ ազատ տոմսը, վաճառի տրված համարով տոմսը, եթե այն ազատ է, և հաշվի յուրաքանչյուր սեկտորում վաճառված տոմսերի քանակը:

Մուտքագրվող տոմսի համարը, սեկտորը և շարքը պետք է լինեն դրական ամբողջ թվեր: Տոմսի գինը պետք է լինի դրական թիվ: Տոմսի վաճառված կամ ազատ լինելը պետք է նշվի համապատասխան արժեքով: Սխալ մուտքագրման դեպքում պետք է արտածել սխալի հաղորդագրություն և կրկին պահանջել մուտքագրել ճիշտ արժեք: Տոմսերի համարները պետք է լինեն տարբեր:

Տեսական մաս. Մշակել ծրագիր, որը կօգտագործի `Ticket` կառուցվածքը և այդ կառուցվածքի 15 տարրից բաղկացած զանգվածը: Հիմնական հաշվարկները, որոնումները և փոփոխությունները կատարել առանձին ֆունկցիաներով: Տվյալների ստուգման համար օգտագործել `while` ցիկլեր և `if` պայմաններ:

- Սահմանում ենք `Ticket` կառուցվածքը՝ անհրաժեշտ դաշտերով:
- Ստեղծում ենք 15 տոմսի համար նախատեսված զանգված:
- Մուտքագրում և ստուգում ենք յուրաքանչյուր տոմսի տվյալները:
- Հաշվում ենք վաճառված տոմսերից ստացված ընդհանուր գումարը:
- Մուտքագրում ենք սեկտորի համարը և արտածում այդ սեկտորի ազատ տոմսերը:
- Գտնում ենք ամենաթանկ ազատ տոմսը:
- Մուտքագրում ենք տոմսի համարը և ստուգում՝ արդյոք այն ազատ է: Եթե ազատ է, կատարում ենք վաճառքը:
- Հաշվում ենք յուրաքանչյուր սեկտորում վաճառված տոմսերի քանակը:
- Արտածում ենք ստացված արդյունքները:

Առաջադրանք. Ստեղծել C++ ծրագիր, որը կմուտքագրի 15 համերգի տոմսերի տվյալները և առանձին ֆունկցիաների միջոցով կհաշվի վաճառված տոմսերից ստացված ընդհանուր գումարը, կարտածի տրված սեկտորի ազատ տոմսերը, կգտնի ամենաթանկ ազատ տոմսը, կվաճառի տրված համարով տոմսը, եթե այն ազատ է, և կհաշվի յուրաքանչյուր սեկտորում վաճառված տոմսերի քանակը: Ծրագիրը պետք է

ստուգի մուտքագրված տվյալների ճշտությունը և սխալ մուտքի դեպքում պահանջի կրկին մուտքագրել տվյալները։

```
#include <iostream>
#include <string>
using namespace std;

const int TICKET_COUNT = 15;

struct Ticket {
   int ticketNumber;
   int sector;
   int row;
   double price;
   bool sold;
};

// Թվային մուտքագրման ստուգում
int readInt(const string& prompt, int minValue, int maxValue) {
   int value;
   while (true) {
      cout << prompt;
      if (cin >> value && value >= minValue && value <= maxValue) {
         return value;
      }
      cout << "Սխալ մուտքագրում։ Փորձեք կրկին։\n";
      cin.clear();
      cin.ignore(10000, '\n');
   }
}
// Տոմսի գնի մուտքագրում
double readPrice(const string& prompt) {
   double value;
   while (true) {
      cout << prompt;
      if (cin >> value && value > 0.0) {
         return value;
      }
      cout << "Գինը պետք է լինի դրական թիվ։\n";
      cin.clear();
      cin.ignore(10000, '\n');
   }
}
// Ստուգել՝ տոմսի համարը եզակի է, թե ոչ
bool isTicketNumberUnique(
   const Ticket tickets[],
   int count,
```

```
    int ticketNumber
) {
    for (int i = 0; i < count; ++i) {
        if (tickets[i].ticketNumber == ticketNumber) {
            return false;
        }
    }
    return true;
}
// Մուտքագրել տոմսերը
void inputTickets(Ticket tickets[], int count) {
    cout << "=== Տոմսերի մուտքագրում ===\n";
    for (int i = 0; i < count; ++i) {
        cout << "\nՏոմս #" << i + 1 << '\n';
        int number;
        while (true) {
            number = readInt(
                "Տոմսի համար: ",
                1,
                2147483647
            );
            if (isTicketNumberUnique(tickets, i, number)) {
                break;
            }
            cout << "Այս համարը արդեն գոյություն ունի։\n";
        }
        tickets[i].ticketNumber = number;
        tickets[i].sector = readInt("Հատված (1-100): ", 1, 100);
        tickets[i].row = readInt("Շարք (1-100): ", 1, 100);
        tickets[i].price = readPrice("Գին: ");
        tickets[i].sold =
            readInt("Վաճառվա՞ծ է (0՝ ոչ, 1՝ այո): ", 0, 1) == 1;
    }
}
// Հաշվել վաճառված տոմսերից ստացված գումարը
double calculateTotalRevenue(
    const Ticket tickets[],
    int count
) {
    double total = 0.0;
    for (int i = 0; i < count; ++i) {
        if (tickets[i].sold) {
            total += tickets[i].price;
        }
    }
    return total;
```

```
}
// Արտածել մեկ տոմսի տվյալները
void printTicket(const Ticket& ticket) {
    cout << "Համար: " << ticket.ticketNumber
         << " | Հատված: " << ticket.sector
         << " | Շարք: " << ticket.row
         << " | Գին: " << ticket.price
         << " | Կարգավիճակ: "
         << (ticket.sold ? "Վաճառված" : "Ազատ")
         << '\n';
}
// Արտածել ընտրված հատվածի ազատ տոմսերը
void displayAvailableTicketsBySector(
    const Ticket tickets[],
    int count,
    int sector
) {
    bool found = false;
    cout << "\nՀատված " << sector << "-ի ազատ տոմսերը:\n";
    for (int i = 0; i < count; ++i) {
        if (tickets[i].sector == sector && !tickets[i].sold) {
            printTicket(tickets[i]);
            found = true;
        }
    }
    if (!found) {
        cout << "Այս հատվածում ազատ տոմսեր չկան:\n";
    }
}
// Գտնել ամենաթանկ ազատ տոմսի ինդեքսը
// Եթե ազատ տոմս չկա, վերադարձնել -1
int findMostExpensiveAvailableTicket(
    const Ticket tickets[],
    int count
) {
    int bestIndex = -1;
    for (int i = 0; i < count; ++i) {
        if (!tickets[i].sold &&
            (bestIndex == -1 ||
             tickets[i].price > tickets[bestIndex].price)) {
            bestIndex = i;
        }
    }
    return bestIndex;
}
```

```
// Վաճառել տոմս ըստ համարի
bool sellTicket(
   Ticket tickets[],
   int count,
   int ticketNumber
) {
   for (int i = 0; i < count; ++i) {
      if (tickets[i].ticketNumber == ticketNumber) {
         if (tickets[i].sold) {
            cout << "Տոմսն արդեն վաճառված է։\n";
            return false;
         }
         tickets[i].sold = true;
         cout << "Տոմսը հաջողությամբ վաճառվեց։\n";
         printTicket(tickets[i]);
         return true;
      }
   }
   cout << "Նշված համարով տոմս չի գտնվել։\n";
   return false;
}
// Հաշվել վաճառված տոմսերը յուրաքանչյուր հատվածում
void countSoldTicketsBySector(
   const Ticket tickets[],
   int count
) {
   bool processed[TICKET_COUNT] = {};
   cout << "\n=== Վաճառված տոմսերն ըստ հատվածների ===\n";
   for (int i = 0; i < count; ++i) {
      if (processed[i]) {
         continue;
      }
      int soldCount = 0;
      for (int j = i; j < count; ++j) {
         if (tickets[j].sector == tickets[i].sector) {
            processed[j] = true;
            if (tickets[j].sold) {
               ++soldCount;
            }
         }
      }
      cout << "Հատված " << tickets[i].sector
           << " — " << soldCount
           << " վաճառված տոմս\n";
   }
}
```

```
// Արտածել բոլոր տոմսերը
void displayAllTickets(
    const Ticket tickets[],
    int count
) {
    cout << "\n=== Բոլոր տոմսերը ===\n";
    for (int i = 0; i < count; ++i) {
        printTicket(tickets[i]);
    }
}
int main() {
    Ticket tickets[TICKET_COUNT]{};
    inputTickets(tickets, TICKET_COUNT);
    int choice;
    do {
        cout << "\n========== ՀԱՄԵՐԳԻ ՏՈՄՍԵՐ ==========\n"
             << "1. Արտածել բոլոր տոմսերը\n"
             << "2. Հաշվել ընդհանուր եկամուտը\n"
             << "3. Ցուցադրել հատվածի ազատ տոմսերը\n"
             << "4. Գտնել ամենաթանկ ազատ տոմսը\n"
             << "5. Վաճառել տոմս ըստ համարի\n"
             << "6. Հաշվել վաճառված տոմսերն ըստ հատվածների\n"
             << "0. Ելք\n";
        choice = readInt("Ընտրեք գործողությունը: ", 0, 6);
        switch (choice) {
            case 1:
                displayAllTickets(tickets, TICKET_COUNT);
                break;
            case 2:
                cout << "Ընդհանուր եկամուտը: "
                     << calculateTotalRevenue(tickets, TICKET_COUNT)
                     << '\n';
                break;
            case 3: {
                int sector = readInt(
                    "Մուտքագրեք հատվածը: ",
                    1,
                    100
                );
                displayAvailableTicketsBySector(
                    tickets,
                    TICKET_COUNT,
                    sector
                );
                break;
            }
```

```
        case 4: {
            int index = findMostExpensiveAvailableTicket(
                tickets,
                TICKET_COUNT
            );
            if (index == -1) {
                cout << "Ազատ տոմսեր չկան։\n";
            } else {
                cout << "Ամենաթանկ ազատ տոմսը։\n";
                printTicket(tickets[index]);
            }
            break;
        }
        case 5: {
            int number = readInt(
                "Մուտքագրեք տոմսի համարը: ",
                1,
                2147483647
            );
            sellTicket(tickets, TICKET_COUNT, number);
            break;
        }
        case 6:
            countSoldTicketsBySector(tickets, TICKET_COUNT);
            break;
        case 0:
            cout << "Ծրագիրն ավարտվեց։\n";
            break;
      }
    } while (choice != 0);
    return 0;
}
```